LSTM Sequence-to-Sequence vs. a Pretrained LLM on Abstractive Summarization

System report — 5 pages excluding references and appendices.

1. Task and dataset

Abstractive single-document summarization on **CNN/DailyMail 3.0.0** (`abisee/cnn_dailymail`, **Apache-2.0**; Hermann et al., 2015; See et al., 2017). The official splits are article-disjoint; we use them unchanged, adding only seeded subsampling for tractability: **79,996** train, **3,000** validation, **11,490** test (mean article 785 tokens, mean summary 55).

Every system is scored on the same 500-article subset of test, drawn once with seed 1234 before any model development, with indices recorded in `dataset_meta.json`.

Leakage was verified, not assumed. Article-ID overlap between train, validation and test is exactly zero; the 500-article set is a verified subset of test disjoint from train; and the four few-shot exemplars actually sent to the LLM are confirmed to come from train.

Pipeline validation. Before comparing anything, we reproduced a published baseline. Our Lead-3 scores **40.04 / 17.50 / 36.34** (ROUGE-1/2/Lsum) on the full test set against See et al.'s published **40.34 / 17.70 / 36.57**. Agreement to ∼0.3 ROUGE indicates the tokenization, sentence splitting, reference construction and ROUGE configuration are correct. Lead-3 is reported throughout: on this dataset it is a famously strong baseline, and a summarization score is uninterpretable without it.

2. System design

2.1 LSTM seq2seq with attention

Implemented directly in PyTorch from `nn.LSTM`, `nn.Embedding`, `nn.Linear` and `nn.Dropout` only — no prebuilt seq2seq framework anywhere in the model, training loop or decoding.

```
Embedding(50,000 × 256, shared) → BiLSTM encoder (256/direction)
  → bridge tanh(W·[h_fwd ; h_bwd]) → Bahdanau attention, masked over padding
  → LSTM decoder (256, input feeding) → tanh(W_c·[h_t ; c_t])
  → output projection (weight-tied to the embedding)
```

15,347,280 parameters, of which 12.8M (83%) is the embedding table — the sequence model itself is ∼2.5M.

Preprocessing. Word-level regex tokenizer, lowercased, with abbreviation-aware sentence splitting. A word-level tokenizer was chosen deliberately over subwords: it makes the OOV failure mode observable, which a subword vocabulary hides by construction. Vocabulary 50,000 types from train only, covering **98.17%** of training tokens. Sources truncated to 400 tokens (following See et al.), targets to 100.

Training. Teacher forcing; label-smoothed cross-entropy (0.1); Adam at 1e-3; gradient clipping 5.0; `ReduceLROnPlateau`; early stopping patience 2; 5 epochs; batch 64 with length-bucketed batching.

Decoding. Beam 4 with the GNMT length penalty and repeated-trigram blocking; `<pad>` and `<unk>` suppressed. Greedy and no-blocking variants are reported as ablations.

Three implementation details mattered, each found by measurement: padding is masked *before* the attention softmax (without it, 63% of attention mass lands on padding for a short article in a mixed batch); the vocabulary projection is applied in time-chunks rather than as one 1.28 GB `(B, T, V)` tensor; and padded shapes are quantized so the Metal backend stops recompiling a kernel per distinct tensor shape — a **34–55× speedup**, and the change that made training feasible. Full accounts in **Appendix E**.

2.2 LLM baseline

Llama 3.1 8B Instruct, 4-bit, run **locally** on the Apple silicon GPU via MLX — the assignment's free open-weights option, so cost is reported as GPU-hours. Greedy decoding, so the baseline is deterministic. This measures the gap

to a *mid-size open model*, a lower bound on the gap to a frontier model; the harness also supports a hosted API backend unchanged.

Two prompt variants, differing on whether the prompt describes the reference *style*: **A ("plain")** is a natural request; **B ("style-matched")** adds the length (~55 words), sentence count (3–4) and register of CNN/DailyMail highlights. Each is run zero-shot and few-shot (k = 4), giving four settings. Exact prompts in **Appendix A**.

Input parity. The LLM receives the article truncated to the **same 400-word window the LSTM encoder sees**. Without parity we would be comparing amounts of input rather than models. This window discards **49% of all article tokens** and truncates **82% of articles** — equally for both systems. We also ran the unmatched condition to check the decision did not handicap the LLM; it did not (§6.3).

3. Experimental settings

Apple M4 Pro (16-core GPU), 24 GB unified memory, macOS 15; PyTorch 2.13 (MPS), Python 3.13. Seed 1234 everywhere. Metric: ROUGE-1/2/Lsum F1 (`rouge-score`, `use_stemmer=True`), 95% percentile bootstrap CIs over 1,000 resamples. Differences between systems use a **paired bootstrap** (10,000 replicates, resampling articles once per replicate and applying the same resample to both systems), because systems scored on the same articles are correlated and independent intervals are too conservative.

4. Results

Full tables in **Appendix F**; per-epoch curves in `runs/*/train_summary.json`.

System	ROUGE-1	ROUGE-2	ROUGE-Lsum	Len
LLM B (style-matched), zero-shot	**41.35** [40.44, 42.20]	**18.07**	**38.14**	82
LLM B, zero-shot, **full article**	40.87 [39.95, 41.70]	17.70	37.65	84
LLM B, few-shot k=4	40.48 [39.62, 41.34]	16.58	37.58	79
Lead-3 baseline	39.89 [38.87, 40.93]	17.60	36.35	86
LLM A (plain), few-shot k=4	39.40 [38.60, 40.21]	14.90	36.15	81
LLM A, zero-shot	38.52 [37.63, 39.34]	14.91	34.65	110
LSTM + attention (beam 4)	**35.00** [34.03, 36.04]	**13.75**	**32.25**	48

4.1 The gap, and what it is made of

Paired bootstrap against the LSTM: the best LLM setting is **+6.35 ROUGE-1** [+5.34, +7.39], $p < 0.0001$, winning on 358 of 500 articles.

But the two prompts differ from each other by 2.83 ROUGE-1 (41.35 vs. 38.52 zero-shot) — **45% of the model gap, from phrasing alone**. Reporting only variant A would have shown a 3.5-point gap; only variant B, 6.4. Both are defensible single numbers and both misattribute specification to capability. This is the strongest argument for testing more than one prompt.

Lead-3 beats four of the five LLM configurations, and on ROUGE-2 loses only to variant B zero-shot, by 0.47. A rule with no parameters outscores an 8B pretrained model unless that model is told what the references look like.

Few-shot helps the weak prompt (+0.88) and hurts the strong one (−0.87). Exemplars and an explicit specification are substitutes: once the target is stated, four examples add variance without information. Variant B's novel-bigram rate rises 0.246 → 0.370 with exemplars — it drifts *away* from the style it was told to adopt.

4.2 Consistency across length and difficulty

The gap is **broadly constant and does not widen with article length** (+7.18 short, +5.13 medium, +6.74 long). This contradicts the obvious hypothesis: if the LSTM's limitation were the recurrent bottleneck compressing long

inputs, the gap should grow. It does not — and the length buckets cannot really test it, since 82% of articles already exceed the encoder window and differ only in how much was discarded before the model saw them.

All systems degrade sharply on abstractive examples (LSTM −9.5 ROUGE-1, LLM −10.6, Lead-3 −10.7) and degrade *together*, which says more about ROUGE than about the models.

4.3 Ablations

Identical hyperparameters, data and seed; exactly one config line differs in each. Paired bootstrap vs. the full model.

Variant	Val PPL	ROUGE-1	Δ	p
LSTM + attention (beam 4)	**35.6**	**35.00**	—	—
— 100-token encoder window	65.5	34.02	−0.98	0.053 *(n.s.)*
— unidirectional encoder	40.0	33.12	−1.87	0.0001
— greedy decoding	35.6	32.60	−2.40	<0.0001
— no trigram blocking	35.6	29.65	−5.35	<0.0001
— **no attention**	**121.7**	**20.97**	**−14.02**	<0.0001

Attention is not a refinement; it is the model. Removing it costs 14.02 ROUGE-1 and collapses ROUGE-2 fourfold (13.75 → 3.47). Unigram overlap at 21 with near-zero bigram overlap is the signature of *summary-shaped text that is not about this article*: 53% of its content words are absent from the source, against 1.5% for the full model. This is the fixed-vector bottleneck — without attention the decoder cannot select *which* part of a 400-token article to describe.

Repetition costs 5.35 ROUGE-1 and is a decoding artifact. Without trigram blocking the duplicate-trigram rate is 0.282; with it, exactly 0.0. The canonical LSTM failure is real but correctable at decoding time, not intrinsic to the trained model.

A 4× smaller encoder window costs nothing measurable ($p = 0.053$; ROUGE-2 $p = 0.495$; win/loss 235/265) despite validation perplexity nearly doubling. The model is measurably worse at *modelling* the text yet produces summaries of indistinguishable quality. Given that the window already discards 49% of tokens, this is strong evidence of lead bias — and why Lead-3 stays competitive.

4.4 Cost, latency, and compute

Measured on the same machine (full table in Appendix F). The LSTM is **15.3M parameters / 61 MB** against ~8B / ~4.5 GB, cost **8.73 GPU-hours** to train once, and generates a summary in **0.231 s** (259/min). The LLM needs **2.85 s** zero-shot (21/min) and 9.2 s few-shot (6.5/min): the LSTM is **12× faster per summary** than the fastest LLM setting — against a quantized 8B model on the *same GPU*, with no network hop. Total LLM generation: **3.36 GPU-hours for 2,000 summaries**; both are $0.00, run locally. Few-shot triples generation cost (2,200 extra prefill tokens per request) for no benefit on the better prompt.

5. Error analysis

Thirteen examples selected **by behaviour, not by score** are in `results/qualitative.md`. Each textbook failure mode below is stated only where the data supports it; one is refuted.

Rare-word breakage (LSTM) — confirmed, but invisible in the obvious metric. The measured OOV rate is **0.000** for every LSTM system — an artifact, since `<unk>` is suppressed so the model *cannot* emit an OOV token (Lead-3, copying real text, shows the true rate of 0.023). The failure relocates to substitution and omission, and **67% of references contain at least one token the model cannot produce**: `fredric brandt` → `dr. frederic brandt` (an in-vocabulary misspelling of a real person, with no metric signature); `teamed up with golfbidder` → `teamed up with the to offer…` (entity dropped, sentence broken). Evidence in Appendix G.

Fluent-but-wrong — confirmed, but only without attention. The no-attention ablation, on a Louisville fire, generated "*the fire is the fire in the city of san diego.*" The full model does not do this (1.5% unsupported content).

"**The LLM hallucinates**" **— refuted.** Unsupported-content is a *lexical* proxy: on constructed cases a faithful paraphrase scored 0.857 while a fabrication reusing source words scored 0.300 (Appendix G.2). Variant B's 0.055 alongside a 0.246 novel-bigram rate is restrained paraphrase, not invention. Establishing real hallucination needs human annotation, which we did not perform.

Format drift (LLM) — confirmed, and its largest weakness here. Variant A zero-shot produces **110-token** summaries against a 58-token reference. Variant B, told "3–4 sentences, ~55 words," produces 82 and gains 2.83 ROUGE-1. The LLM's headline failure is not faithfulness but *failing to infer an unstated output specification*.

The two fail in opposite directions. The LSTM's errors are omission — it copies (novel-bigram 0.080), stays short (48 vs. 58 tokens), drops what it cannot say. The LLM's are excess — it rewrites (0.246), over-produces (82), adds correct-but-unrequested detail. In one example the LSTM *beats* the LLM 52.6 to 42.0 precisely because the LLM added true information the reference omitted. ROUGE rewards the LSTM's direction more than its quality warrants.

6. Discussion

6.1 Why the gap exists

Pretraining and transfer dominate. The LSTM sees 80k pairs and nothing else, and must learn English, news register and the summarization objective at once — with 83% of its parameters spent on an embedding table. That the LLM reaches 41.35 with no gradient step on this data *is* the transfer-learning result. **Capacity matters less than the counts suggest**: a 500× parameter difference yields an 18% relative ROUGE-1 difference.

The recurrent bottleneck binds only when attention is absent. Removing attention costs 14.02 ROUGE-1 — that is the bottleneck, and self-attention solves the same problem more thoroughly. But a 4× smaller window costs nothing significant and the gap does not widen with length: on this dataset the LSTM fails at knowing what a sentence *means*, not at carrying information across 400 timesteps. **Specification accounts for a further 45% of the gap** (§4.1), which no architectural account explains.

6.2 Failure-mode contrast

Novel-bigram rate 0.080 (LSTM) vs. 0.246 (LLM); unsupported content 0.015 vs. 0.055; length 48 vs. 82 tokens against a 58-token reference. 92% of the LSTM's bigrams appear verbatim in the source: it is closer to a learned extractive system than an abstractive one. The LLM produces 41% more text than the reference and rewrites a quarter of it. **ROUGE is not neutral between these** — it rewards copying and penalises paraphrase. That the LLM wins anyway suggests the true quality gap exceeds 6.35 points, while Lead-3, which is pure copying, outscores most LLM settings.

6.3 Fairness of the comparison

Unfair **to the LSTM**: the LLM was pretrained on a corpus larger by many orders of magnitude, and CNN/DailyMail is among the most widely mirrored NLP benchmarks, so its "zero-shot" performance may partly reflect memorization.

Unfair **to the LLM**: the LSTM is trained on this dataset's reference distribution and so optimizes the exact stylistic target ROUGE rewards, while the LLM must be told about it through a prompt. We can quantify this: the A-vs-B gap of **2.83 ROUGE-1** is the penalty for not being told, and it is 45% of the total gap.

Input parity does not handicap the LLM — we measured it. Given the *untruncated* article, variant B zero-shot scores **40.87** vs. **41.35** on the matched window: **−0.47 ROUGE-1** [−0.91, −0.03], $p = 0.034$. More input made it slightly *worse*. This is consistent with §4.3 and with Lead-3's strength — the summary-relevant content sits in the opening — and it means the parity choice costs the LLM nothing.

A fairer middle point is fine-tuning a small pretrained transformer (BART-base, T5-small) on the same 80k pairs, isolating pretraining from architecture and supervision. Our results predict it would close most of the gap, receiving both.

6.4 Engineering trade-offs

Latency: 0.231 s vs. 2.85 s — 12×, against a quantized 8B model on the same GPU with no network hop; any sub-second interactive budget excludes the LLM before cost is considered. **Throughput:** 259 vs. 21 summaries/min — at a million documents, ~2.7 GPU-days versus 33, with the 8.73-hour training cost repaid after ~40,000 documents. **Deployment:** 61 MB versus 4.5 GB — commodity CPU, embedded hardware, or an air-gapped network. **Controllability:** our failure modes are bounded and fixable at decoding (repetition eliminated, length governed by the GNMT penalty, `<unk>` suppressed), whereas the LLM ignored an explicit length instruction by 41% and the remedy is prompt iteration with no guarantee. **Low-resource settings:** the LLM's advantage comes from pretraining, so where that pretraining does not exist it disappears, while 80k supervised pairs remains achievable.

Where the LLM clearly wins: any task with no training data. It scored 41.35 having never seen this dataset, and we have no counterpart to that.

6.5 Limitations and ethics

The models are undertrained, and we can bound it. All four runs hit the 5-epoch cap with validation loss still strictly improving; early stopping never fired. The per-epoch perplexity gain halved consistently (335.8 → 70.0 → 47.0 → 39.4 → 35.6), extrapolating to ~31.8 — roughly **11% of remaining headroom unclaimed**. That would not close a 6.35-point gap, but the LSTM numbers are a floor, not a ceiling.

Half the article is discarded. The 400-token window removes 49% of all article tokens and truncates 82% of articles — identically for both systems, so the comparison is unbiased, but neither system is evaluated on full-document summarization.

Metric limitations. ROUGE measures n-gram overlap with a single reference. It rewards extractive copying — which is why Lead-3 scores 39.89 — and cannot distinguish a factually wrong summary from a correct paraphrase. Our unsupported-content diagnostic is a lexical proxy, not a factuality judgement. No human evaluation was performed.

Dataset bias and licensing. English-only US/UK news from a narrow period, whose "summaries" are editor-written highlight bullets. Conclusions do not transfer to other languages, genres or summary styles. Apache-2.0; the articles remain the publishers' property and are not redistributed.

Contamination risk. We cannot verify what was in the LLM's pretraining data. Its numbers should be read as an upper bound on genuinely held-out performance. Our few-shot exemplars come from train, which controls the leakage we can.

Compute. 8.73 GPU-hours of training plus 3.36 of generation — small absolutely, but the LLM's pretraining cost is amortized across all users and invisible here, a real asymmetry when comparing "compute used".

7. Conclusion

A 15.3M-parameter LSTM with attention, trained from scratch for 8.73 GPU-hours on 80k pairs, reaches **35.00 ROUGE-1**. A 4-bit Llama 3.1 8B, given the same 400-word window and no training on this dataset, reaches **41.35**. The gap is real ($p < 0.0001$) and it is **6.35 points, not an order of magnitude**.

Three results complicate the expected narrative. **Prompt phrasing accounts for 45% of the gap** — much of what looks like capability is specification. **Lead-3 outscores four of the five LLM configurations**, which says more about ROUGE than about either system. And **the recurrent bottleneck is not the binding constraint here**: removing attention costs 14 ROUGE-1, but a 4× smaller encoder window costs nothing measurable.

The engineering conclusion stands regardless: the LSTM is 12× faster, 74× smaller, runs offline, and has bounded failure modes fixable at decoding time. For a task with training data, a latency budget or a data-residency constraint it remains defensible. For a task with *no* training data it has no answer — and that, rather than six ROUGE points, is what pretraining bought.

8. References

- Hermann, K. M., Kočiský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). Teaching Machines to Read and Comprehend. *NeurIPS*.
- See, A., Liu, P. J., & Manning, C. D. (2017). Get To The Point: Summarization with Pointer-Generator Networks. *ACL*.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*.
- Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *EMNLP*.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS*.
- Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Text Summarization Branches Out*.
- Wu, Y., et al. (2016). Google's Neural Machine Translation System. *arXiv:1609.08144*. (GNMT length penalty.)

Appendix A — Exact prompts

`[[FILL: paste verbatim from src/llm/prompts.py and the run .meta.json files — both system prompts, both user templates, and the four few-shot exemplars]]`

Appendix B — Contribution statement

The project decomposes into five workstreams of comparable weight. Fill in the name against the workstream each member actually owned, and adjust the wording where the real division differed — a contribution statement is only useful if it is accurate.

Group **CP-468-D — AI — 17**.

Member	Workstream	Specific responsibilities
Mohanad Bahammam	**Data & preprocessing**	Dataset acquisition and licensing check; frozen seeded splits and the shared head-to-head subset; word-level tokenizer, normalization, and abbreviation-aware sentence splitting; train-only vocabulary construction and coverage analysis; leakage controls. Files: `src/data/*`.
Yakup Bastug	**Model implementation**	BiLSTM encoder and the bridge to the decoder state; Bahdanau, Luong, and no-attention modules with source masking; LSTM decoder with input feeding; weight tying; greedy and beam search with length penalty and trigram blocking. Files: `src/models/*`.
Orhan Gundogan	**Training & performance**	Training loop, label smoothing, bucketed batching, early stopping and LR scheduling; the four experiment configs; the memory and MPS-shape performance work described in §2.1. Files: `src/train.py`, `configs/*`, `scripts/train_all.sh`.
Ayuub Hagi	**LLM baseline**	Prompt variant design and the zero-/few-shot protocol; the local MLX backend and the API backend; input-parity design; token, latency, and compute accounting; resumability. Files: `src/llm/*`.
Khaled Mobarak	**Evaluation, analysis & report**	ROUGE harness with bootstrap CIs; Lead-3 validation against the literature; length and abstractiveness bucketing; repetition/OOV/unsupported-content diagnostics; qualitative selection and error categorization; this report. Files: `src/evaluate.py`, `src/qualitative.py`, `scripts/collect_results.py`, `reports/*`.

Per-member ownership detail — the files each member owns and the results they are accountable for — is in `TEAM.md` in the repository.

Every member should additionally be able to explain the whole pipeline end to end; the demo (Appendix D) exercises all five workstreams in one run.

Appendix C — AI-use disclosure

Tools and use. During this project we used **Claude (Anthropic)** for the following: (1) the initial model, training loop, and evaluation scripts; (2) suggesting evaluation edge cases and diagnostics; (3) report review.

Results. We did **not** use AI to generate our experimental results. Every number reported here was produced by executing the committed code on our own hardware; `scripts/collect_results.py` reads the run artifacts directly and will not emit a figure whose artifact is missing, so all results are reproducible from the repository by anyone.

Responsibility. All AI-assisted code was reviewed and tested by the authors, who ran the full pipeline end to end and are responsible for its correctness. Each author owns a specific component and can account for its design decisions (see Appendix B and `TEAM.md`).

LLM as object of study. Separately from the above, an LLM is the *subject* of the experiment: Llama 3.1 8B Instruct is the baseline system in §2.2. That use is methodological rather than authorial and is documented in full in §2.2 and §4, with the exact prompts in Appendix A.

Appendix D — Repository and demo

- Code: https://github.com/KhaledM0barak/lstm-vs-llm-summarization
- Demo video (8 min): `[[VIDEO URL]]`
- Testing and recording runbook: `DEMO.md`

Appendix E — Implementation notes

Three implementation details from §2.1, in full. Each was found by measuring rather than by reasoning about the code, and two of them were the difference between a run that finishes and one that does not.

1. Attention masking. Padding is masked before the attention softmax. Without the mask the decoder places probability mass on `<pad>` for every short article batched with long ones, silently corrupting the context vector rather than raising an error.

2. Chunked vocabulary projection. Projecting decoder states to the 50k vocabulary in a single matmul materializes a `(B, T, V)` tensor — 1.28 GB in fp32 at batch 64 and 100 steps — which a mask-indexed loss then copies twice more. Peak memory drove the 24 GB machine 9.5 GB into swap and degraded throughput from 1.1 s/batch to 3.2 s/batch and worsening. Applying the projection in 16-step chunks with `F.cross_entropy` (native label smoothing and `ignore_index`, so no masked copies are materialized) holds peak RSS at 1.15 GB.

3. Padded-shape quantization. The dominant cost, and the least obvious. PyTorch's Metal (MPS) backend compiles a kernel per distinct tensor shape, and length-bucketed batching produces a near-unique `(batch, src_len, tgt_len)` triple almost every step. Training was therefore spending most of its wall-clock in shader compilation — `MTLCompilerService` pinning two cores — rather than in arithmetic. Rounding padded lengths up to multiples of 64 (source) and 16 (target), and dropping each pool's ragged final batch to fix the batch dimension, collapses thousands of shapes into a few dozen that compile once and are reused:

Configuration	Before	After	Speedup
Bahdanau attention + input feeding	2537 s	73.7 s	**34×**
Multiplicative attention, batched	2695 s	49.1 s	**55×**

(one epoch over 3,840 examples, batch 64, identical hardware)

The cost is under 2% of training examples per epoch and a little extra padding. This is an accelerator-specific detail rather than anything about summarization, but it is the kind of finding that only appears if throughput is measured rather than assumed — and without it this project's four training runs would have taken an estimated 20+ hours instead of 8.7.

A related failure mode, found while building the demo. The Metal backend does not raise when a command buffer runs out of GPU memory; it returns whatever was in the buffer. With a local LLM resident on the GPU, the LSTM silently produced empty or degenerate `the the the a the` output — indistinguishable from a broken model. `src/demo.py` therefore decodes a known article at startup, checks the output's unique-token ratio, and falls back to CPU with a warning if it looks degenerate.

Appendix F — Full result tables

Generated by `python scripts/collect_results.py`, which reads the run artifacts directly; see also `results/results.md`.

F.1 All systems (ROUGE F1 ×100, 95% bootstrap CI, n = 500)

System	ROUGE-1	ROUGE-2	ROUGE-Lsum	Len
LLM B, zero-shot	41.35 [40.44, 42.20]	18.07 [17.25, 18.84]	38.14	82.1
LLM B, zero-shot, full article	40.87 [39.95, 41.70]	17.70 [16.92, 18.50]	37.65	84.0
LLM B, few-shot k=4	40.48 [39.62, 41.34]	16.58 [15.82, 17.36]	37.58	79.1
Lead-3 baseline	39.89 [38.87, 40.93]	17.60 [16.58, 18.60]	36.35	86.2
LLM A, few-shot k=4	39.40 [38.60, 40.21]	14.90 [14.23, 15.58]	36.15	81.1
LLM A, zero-shot	38.52 [37.63, 39.34]	14.91 [14.28, 15.56]	34.65	109.9
LSTM + attention (beam 4)	35.00 [34.03, 36.04]	13.75 [12.85, 14.64]	32.25	48.2
— 100-token window	34.02 [33.10, 34.91]	13.44 [12.59, 14.29]	31.49	41.8
— unidirectional	33.12 [32.21, 34.08]	12.64 [11.84, 13.49]	30.37	45.8
LSTM (greedy)	32.60 [31.66, 33.62]	12.05 [11.25, 12.85]	30.57	48.3
LSTM (no trigram block)	29.65 [28.69, 30.71]	10.92 [10.14, 11.76]	26.99	51.8
— no attention	20.97 [20.32, 21.67]	3.47 [3.15, 3.81]	19.40	38.2

F.2 Paired bootstrap vs. LSTM + attention (10,000 replicates)

System	Δ ROUGE-1 [95% CI]	p	Δ ROUGE-2	p	W/L
LLM B, zero-shot	+6.35 [+5.34, +7.39]	<0.0001	+4.33	<0.0001	358/142
LLM B, few-shot	+5.48 [+4.43, +6.54]	<0.0001	+2.83	<0.0001	348/152
Lead-3	+4.90 [+3.85, +5.94]	<0.0001	+3.86	<0.0001	333/167
LLM A, few-shot	+4.40 [+3.42, +5.41]	<0.0001	+1.16	0.014	329/171
LLM A, zero-shot	+3.52 [+2.52, +4.55]	<0.0001	+1.17	0.012	314/185
— 100-token window	−0.98 [−1.96, +0.02]	0.053 *(n.s.)*	−0.31	0.495 *(n.s.)*	235/265
— unidirectional	−1.87 [−2.74, −0.98]	0.0001	−1.11	0.003	210/284
LSTM (greedy)	−2.40 [−3.23, −1.53]	<0.0001	−1.69	<0.0001	204/291
LSTM (no trigram block)	−5.35 [−6.03, −4.67]	<0.0001	−2.83	<0.0001	62/330
— no attention	−14.02 [−15.02, −13.00]	<0.0001	−10.28	<0.0001	56/442

F.2b Input-parity check: full article vs. matched 400-word window

Paired bootstrap, variant B zero-shot, n = 500.

Metric	Δ (full − matched)	95% CI	p	W/L
ROUGE-1	−0.47	[−0.91, −0.03]	0.034	177/195
ROUGE-2	−0.37	[−0.81, +0.07]	0.099 *(n.s.)*	164/201
ROUGE-Lsum	−0.49	[−0.93, −0.05]	0.028	172/198

F.3 Behavioral diagnostics (means)

System	Dup-trigram	Novel-bigram	Unsupported	OOV	Empty
LSTM + attention	0.000	0.080	0.015	0.000	0.000
LSTM (greedy)	0.000	0.218	0.065	0.000	0.000
LSTM (no trigram block)	0.282	0.061	0.008	0.000	0.000
— no attention	0.000	0.704	0.530	0.000	0.000
— unidirectional	0.000	0.088	0.024	0.000	0.000
— 100-token window	0.000	0.150	0.040	0.000	0.000
LLM A, zero-shot	0.007	0.521	0.206	0.021	0.000
LLM A, few-shot	0.005	0.500	0.177	0.021	0.000
LLM B, zero-shot	0.009	0.246	0.055	0.029	0.000
LLM B, few-shot	0.010	0.370	0.100	0.024	0.000
Lead-3 baseline	0.011	0.000	0.000	0.023	0.000

OOV is 0.000 for every LSTM system **by construction** — `<unk>` is suppressed at generation, so the model cannot emit an out-of-vocabulary token. See §5.

F.4 ROUGE-1 by source length and by reference abstractiveness

System	Short (≤534)	Medium (534–834)	Long (>834)
LLM B, zero-shot	42.87	41.29	39.89
Lead-3	42.00	40.81	36.87
LSTM + attention	35.69	36.16	33.15
— 100-token window	35.57	35.37	31.14
— no attention	21.71	20.40	20.81

System	Extractive	Mixed	Abstractive
LLM B, zero-shot	46.29	42.09	35.67
Lead-3	45.35	39.72	34.61
LSTM + attention	40.04	34.44	30.52
— no attention	22.15	21.46	19.32

F.5 Training and inference cost

Run	Parameters	Epochs	GPU-hours	Best val loss	Val PPL
base	15,347,280	5	3.05	4.6219	35.6
no_attention	15,150,416	5	2.67	5.6369	121.7
unidirectional	14,558,800	5	1.90	4.7204	40.0
short_context	15,347,280	5	1.11	5.1179	65.5

Total training **8.73 GPU-hours**. LSTM inference: 500 summaries in 115.6 s = **0.231 s each** (259/min), model 61 MB in fp32.

LLM setting	Input tok	Output tok	Wall-clock	Throughput
A zero-shot	270,587	62,501	25.2 min	19.9 / min
B zero-shot	318,087	48,387	23.7 min	21.1 / min
A few-shot k=4	1,234,587	48,433	76.5 min	6.5 / min
B few-shot k=4	1,328,087	47,914	76.5 min	6.5 / min

Total LLM generation **3.36 GPU-hours** for 2,000 summaries, monetary cost **$0.00** (run locally).

Appendix G — Error-analysis evidence

The claims in §5 are verified against the data rather than asserted. This appendix reproduces the evidence. Full side-by-side comparisons for all thirteen behaviour-selected examples are in `results/qualitative.md`.

G.1 The OOV failure is real but invisible in the OOV metric

`<unk>` is suppressed at generation, so every LSTM system reports an OOV rate of exactly 0.000. Lead-3, which copies real article text, shows the true rate for this corpus (0.023). **334 of 500 references (67%) contain at least one token outside the 50,000-type vocabulary**, so the failure necessarily surfaces as substitution or omission instead:

Reference contains	Model produced	Failure
`fredric brandt`	`dr. frederic brandt`	In-vocabulary **misspelling of a real person** — a factual error with no metric signature
`sportsmail have teamed up with golfbidder`	`sportsmail have teamed up with the to offer…`	Entity **dropped**, sentence left ungrammatical

G.2 Unsupported-content is a lexical proxy, not a factuality measure

Constructed cases against a fixed source article ("*the mayor announced on tuesday that the city will build a new hospital in the eastern district at a cost of forty million dollars*"):

Candidate summary	Truth	Unsupported-content
"the city plans to construct a medical facility costing $40m"	**correct**	**0.857**
"the mayor announced the hospital will be built in the **western** district at a cost of **four** million dollars"	**false**	**0.300**
"the mayor announced on tuesday that the city will build a new hospital"	correct (verbatim)	0.000

The faithful paraphrase scores nearly **three times worse** than the fabrication. The metric counts lexical novelty, which correlates weakly with factual error and here inverts. This is why §5 declines to call the LLM's 0.055 rate a hallucination rate.

G.3 What the attention mask prevents

Measured on one short article (3 real tokens) batched with a longer one, using the trained model's attention module:

```
WITHOUT mask: 0.132 0.094 0.146 | 0.108 0.107 0.144 0.146 0.123
WITH mask   : 0.354 0.253 0.393 | 0.000 0.000 0.000 0.000 0.000
                 real tokens    |            padding
```

62.8% of attention mass lands on padding without the mask, and the resulting context vector differs by 1.83 (L2). It also makes the model's output depend on *padding content* — i.e. on which other articles happen to share its batch — which is a correctness violation, not merely a quality loss. `tests/test_models.py` asserts both properties.

G.4 The two systems fail in opposite directions

Largest LLM advantage (ROUGE-1 11.3 → 53.6). The reference concerns a beauty-pageant dispute; the LSTM latches onto a prayer quoted mid-article and misses both principals, while the LLM identifies who said what.

> **Reference:** indianara carvalho posted photo of body painted with image of virgin mary . runner-up claudia alende has branded winner an 'attention-seeking sl*t' …
>
> **LSTM (R1 11.3):** claudia james, 21, came second in last year's beauty pageant. she was accompanied with a prayer which including the words: 'good friday. lord, on this day i ask for peace, love, wisdom and strength'.
>
> **LLM (R1 53.6):** Claudia Alende, 21, launched a scathing attack on Miss Bumbum winner Indianara Carvalho, calling her an 'attention-seeking sl*t'. Indianara Carvalho, 23, posted a picture of her naked body painted with an image of the Virgin Mary on Good Friday…

LSTM beats the LLM (57.1 vs. 26.3) — and the mechanism is instructive. The LSTM copies the reference's exact framing; the LLM writes a longer summary that is **entirely true** but reports different facts (follower counts, likes, due date) than the reference chose.

> **Reference:** sarah stage, 30, has documented her changing figure via her instagram page throughout her pregnancy …
>
> **LSTM (R1 57.1, 28 tokens):** sarah stage, a 30-year-old underwear model and animal rights activist from los angeles, has documented her changing figure via her instagram page throughout her pregnancy.
>
> **LLM (R1 26.3, 85 tokens):** Sarah Stage, a 30-year-old underwear model, has shared a photo of her barely-there baby bump 10 days before her due date. The model, who has 1.5 million Instagram followers, posted the picture on Monday…

Neither output is wrong. The LSTM scores twice as high because it happened to select the same facts the editor did, which is what ROUGE measures.